Ingeniería de soluciones con AI

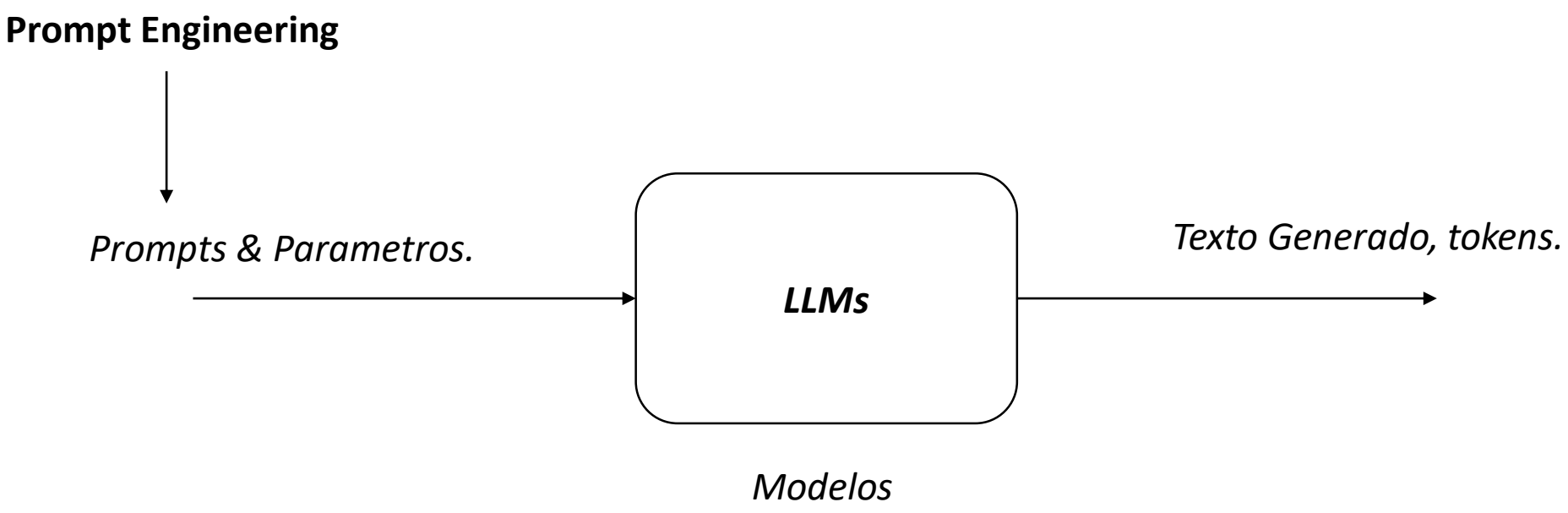
Semestre 2026-1

Docente:

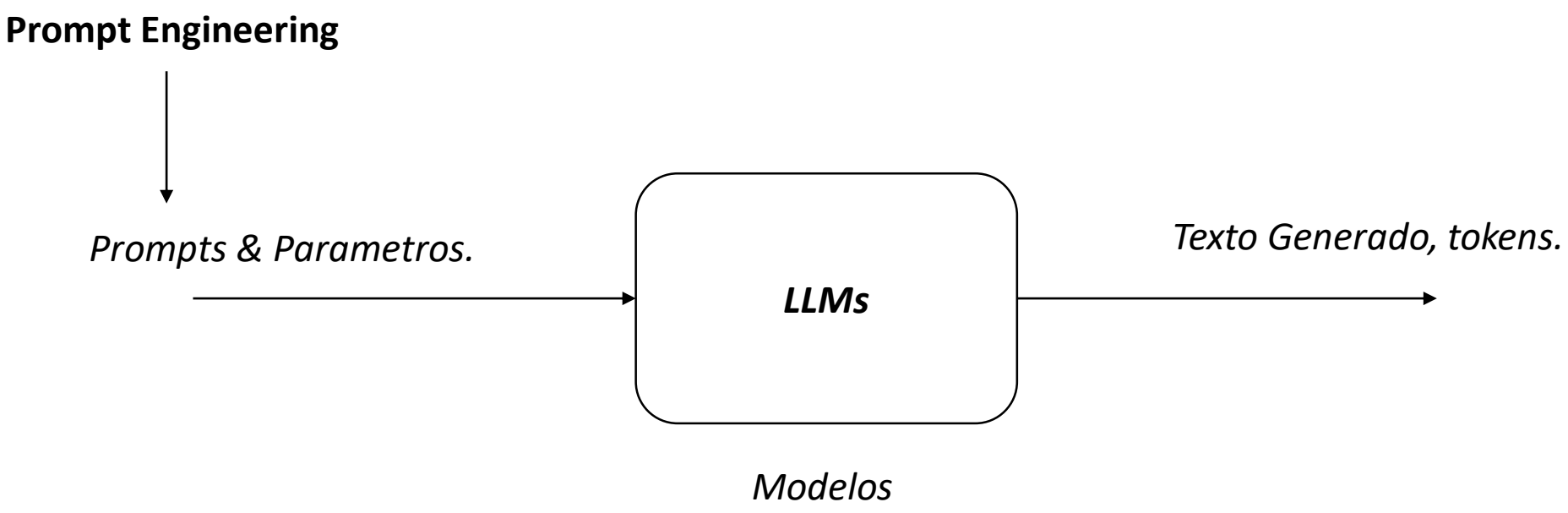
Francisco Macaya

Noticias: AI no es solo software, sino hardware.

Recapitulemos: *Los LLMs como un sistema aislado.*

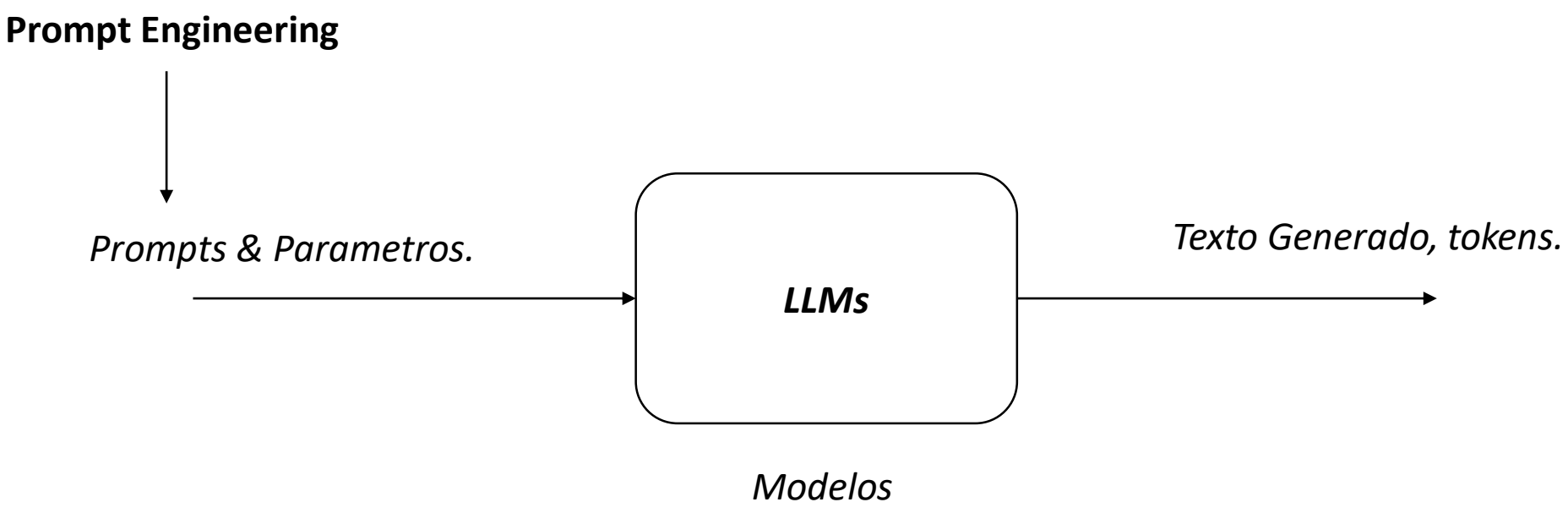


Recapitulemos: *Los LLMs como un sistema aislado.*



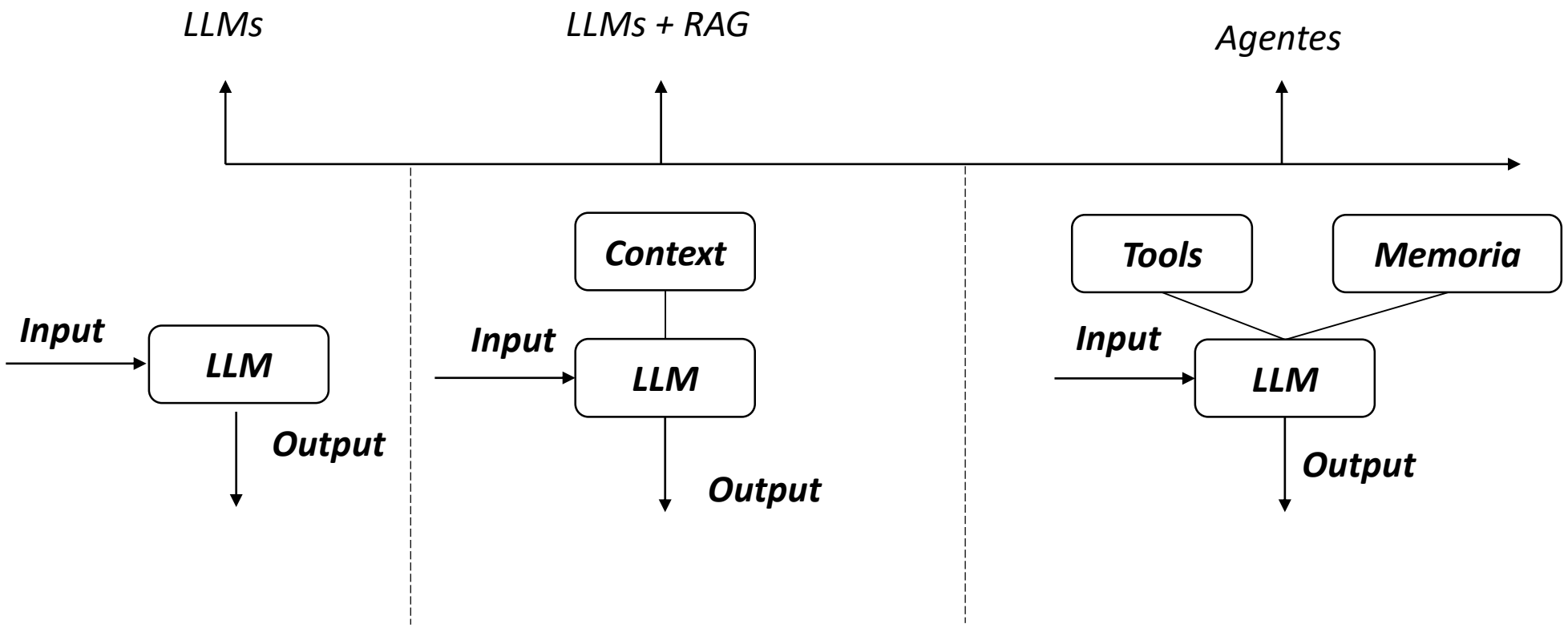
¿Qué falta?

Recapitulemos: *Los LLMs como un sistema aislado.*

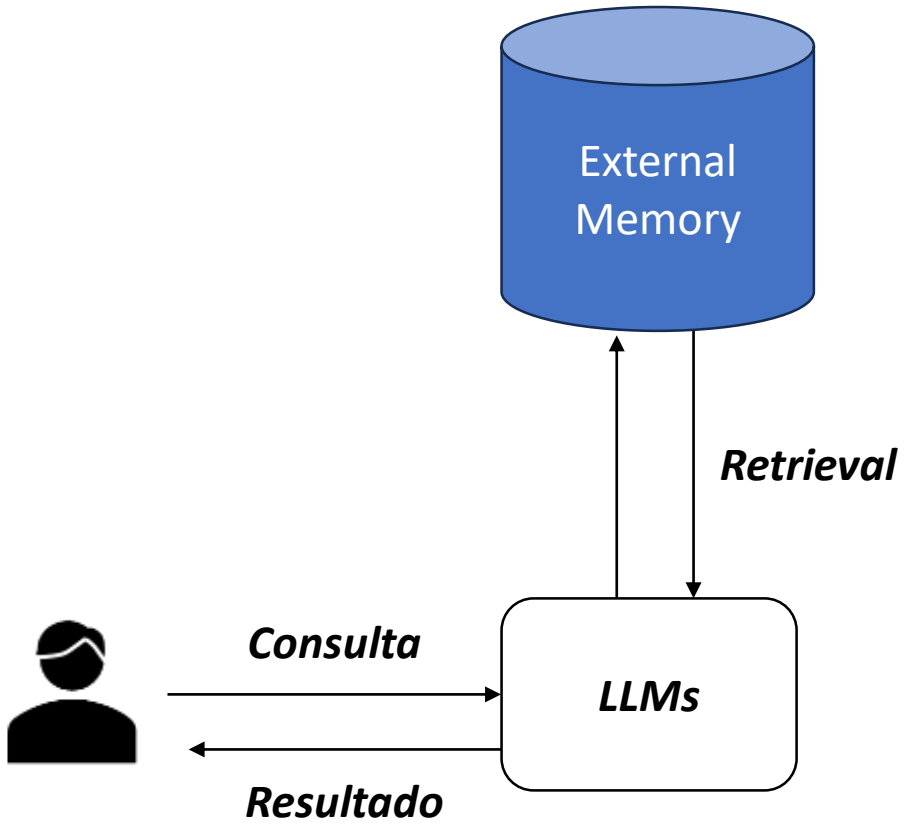


Los modelos no tienen acceso a datos actuales ni privados. No tienen memoria . No tienen acceso aplicaciones, ni al mundo real. No pueden ejecutar algo. Son como un niño que solo me sirve para tareas generalistas.

De generalista a especialista: El camino de la AI.



¿Qué es RAG?: *Retrieval-Augmented Generation*



RAG es una metodología para dotar a los LLMs con información que no tenían al momento de entrenarlos.

1. ***Retrieval (Recuperar):*** *Encontrar las fuentes relevantes a la pregunta.*
2. ***Aumentar (Augment):*** *La información recuperada se inyecta al prompt.*
3. ***Generar (Generative):*** *El LLM produce una respuesta en base a la información dada.*

Retrieval: *Dos caminos, mismo objetivo.*

Term base retrieval

Term Frecuency (TF)*:* Numero de veces que un termino aparece en un documento. Si el termino aparece más veces, es más relevante.

Inverse document frecuency (IDF) *:* La importancia de un termino es inversamente proporcional a la cantidad de documentos donde aparece el termino.

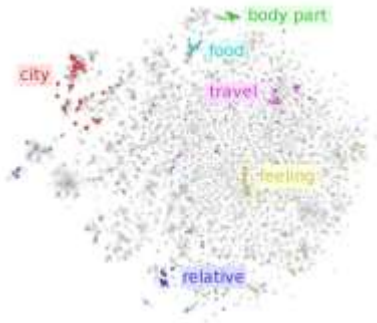
Las soluciones más típicas a este tipo de retrieval son: ***ElasticSearch*** *y* ***BM25****.*

Esta solución se enfoca en el lexical level

Embedding base retrieval

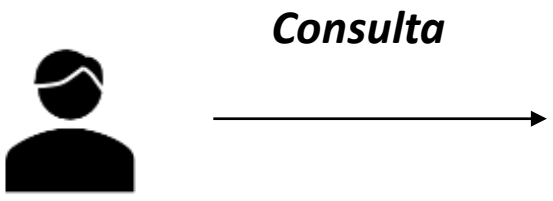
Transforma la información en vectores, se encuentran los más parecidos, y luego se inyectan al modelo de lenguaje como parte del prompt.

Los vectores son capaces de capturar la representación semántica del texto.

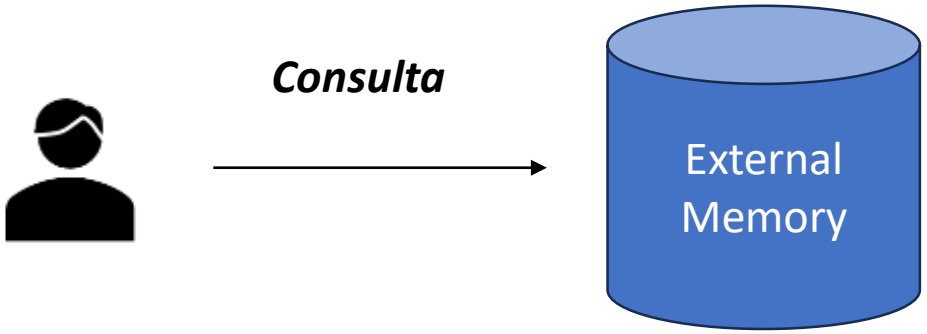


Esta solución se enfoca en el semantic level.

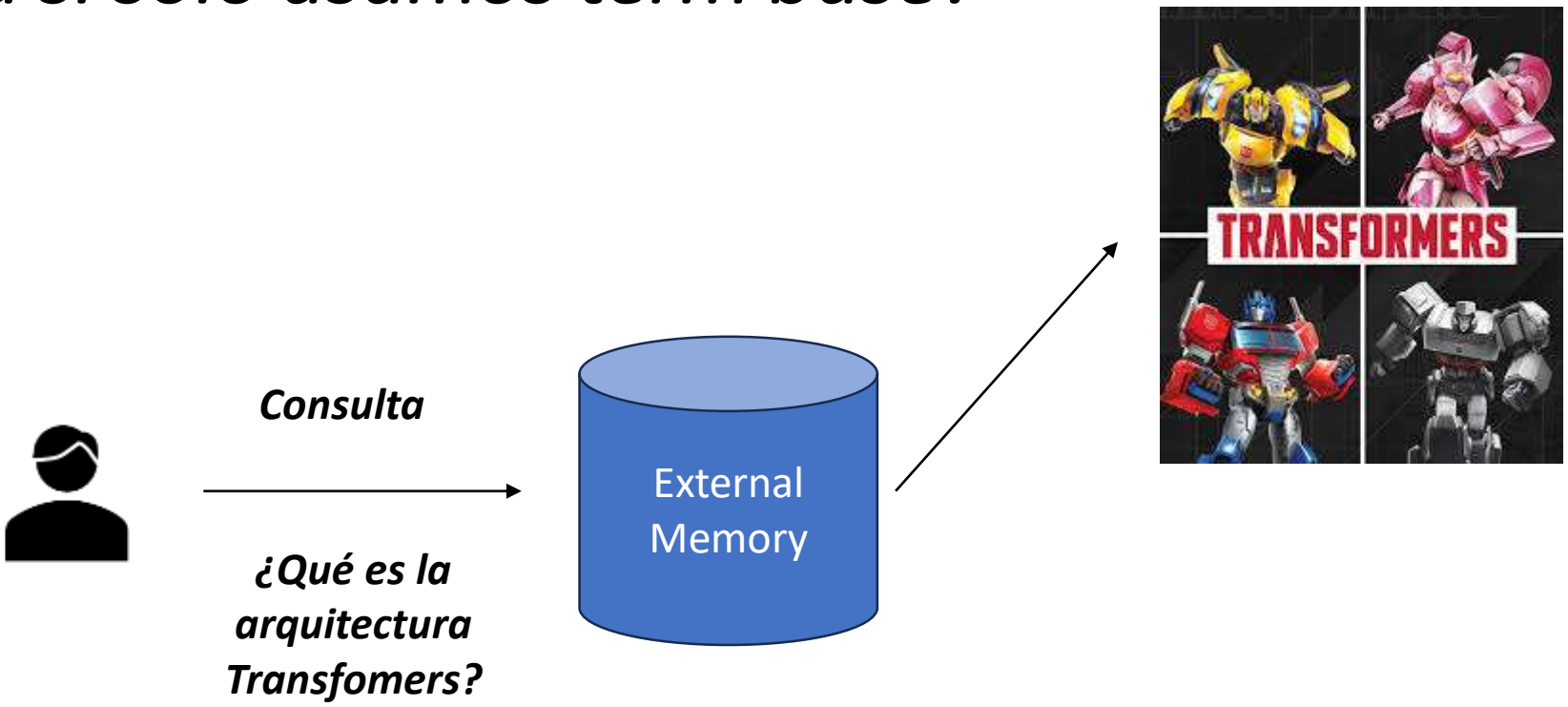
¿Qué pasaría si solo usamos term base?



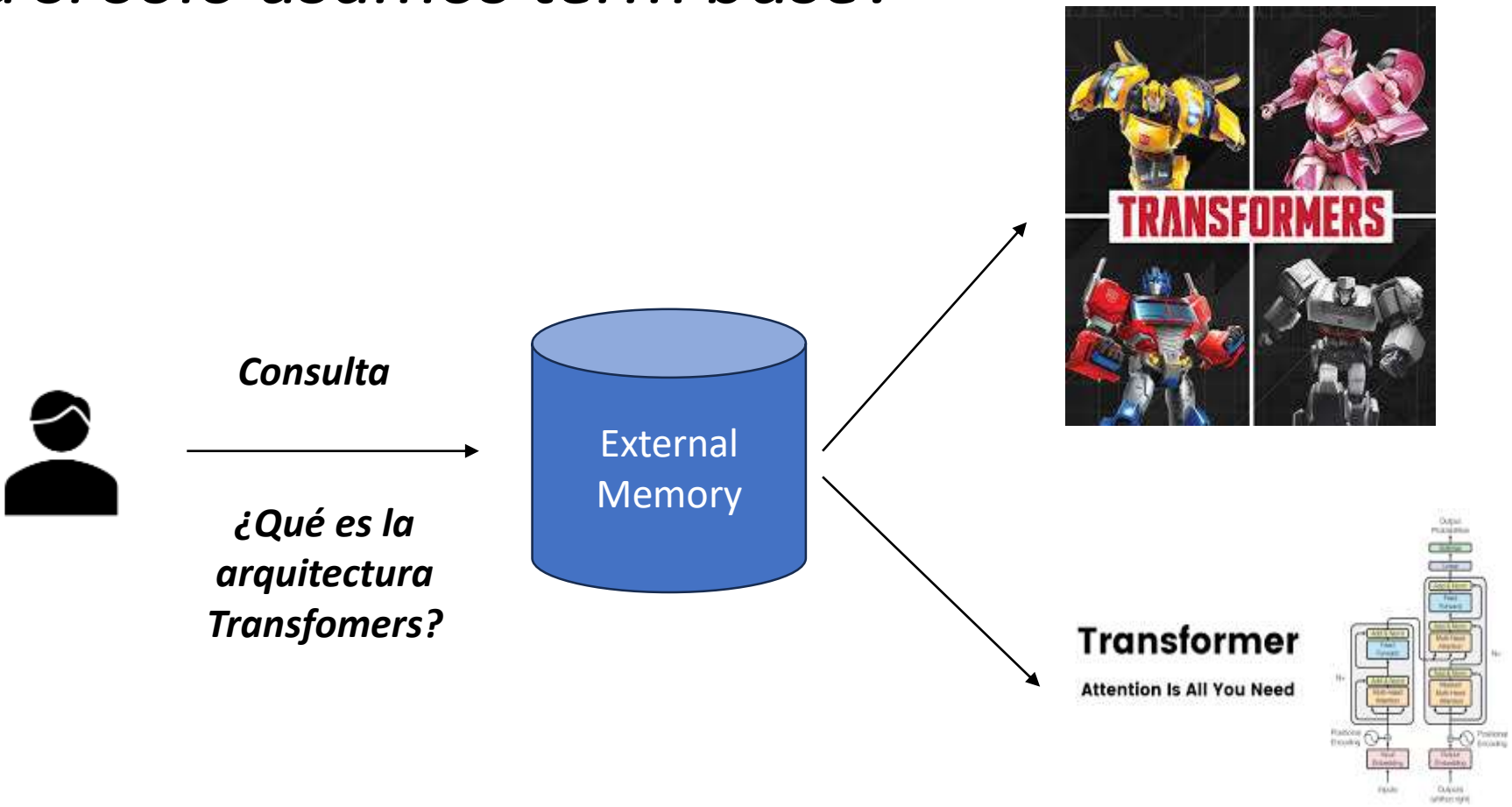
¿Qué pasaría si solo usamos term base?



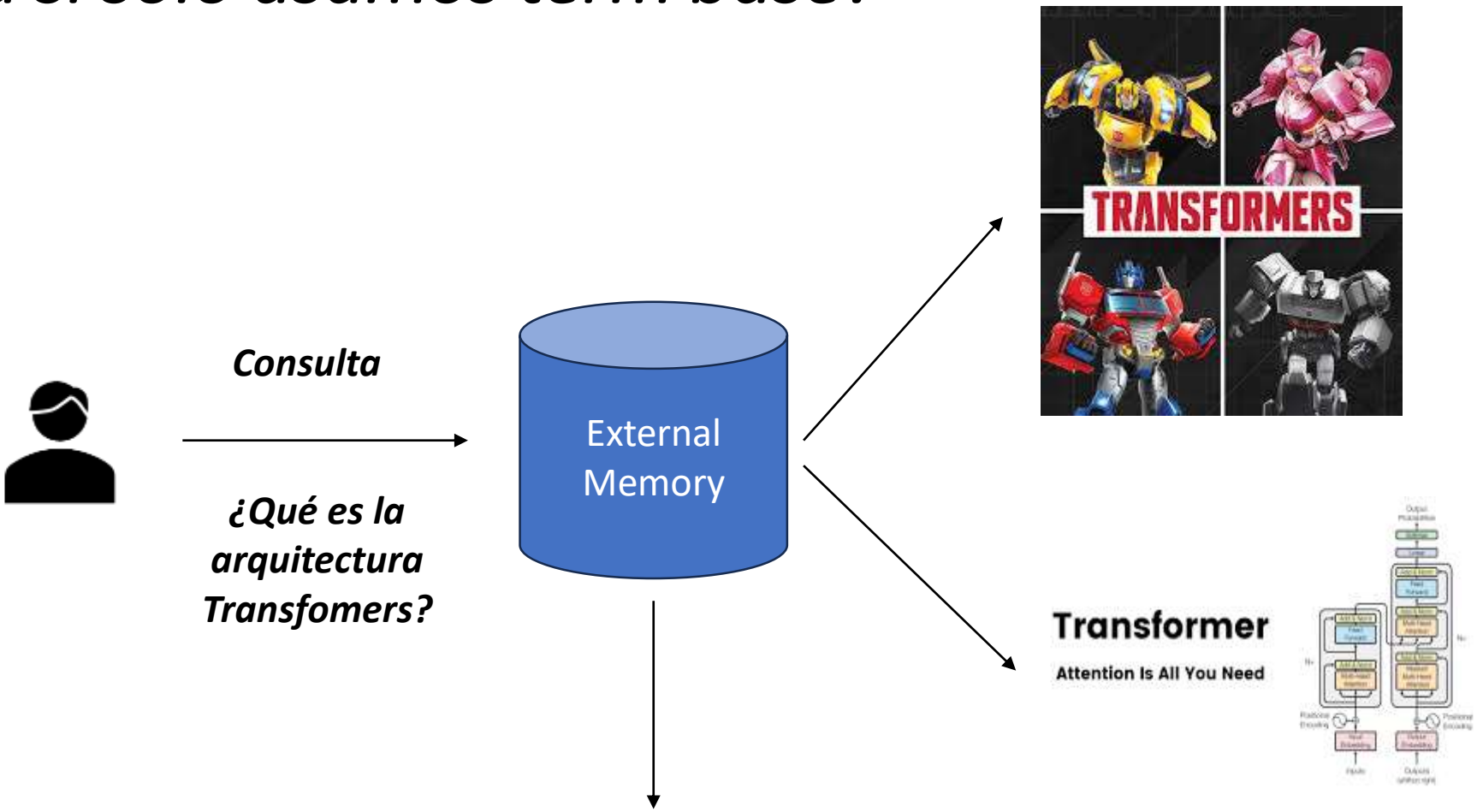
¿Qué pasaría si solo usamos term base?



¿Qué pasaría si solo usamos term base?

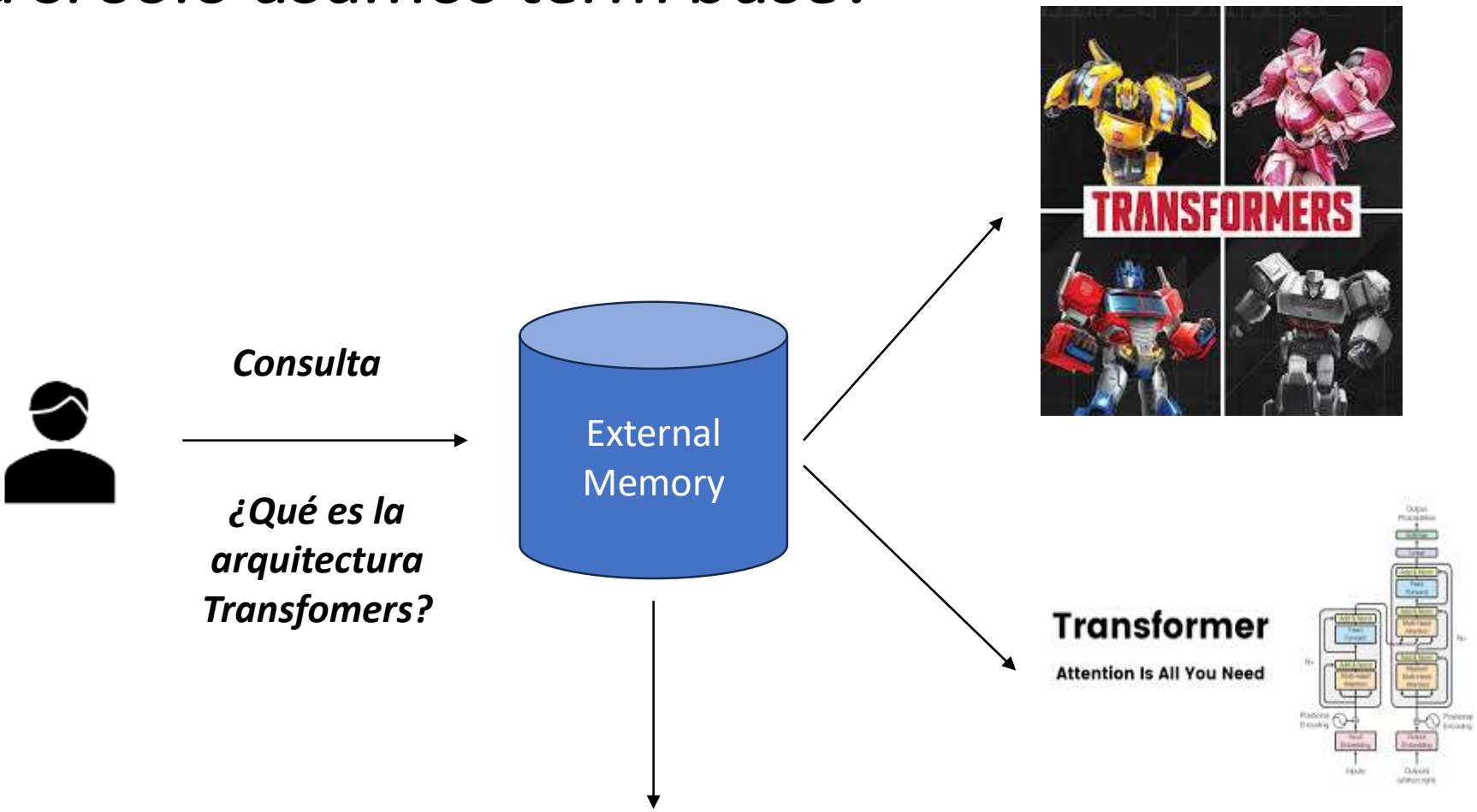


¿Qué pasaría si solo usamos term base?



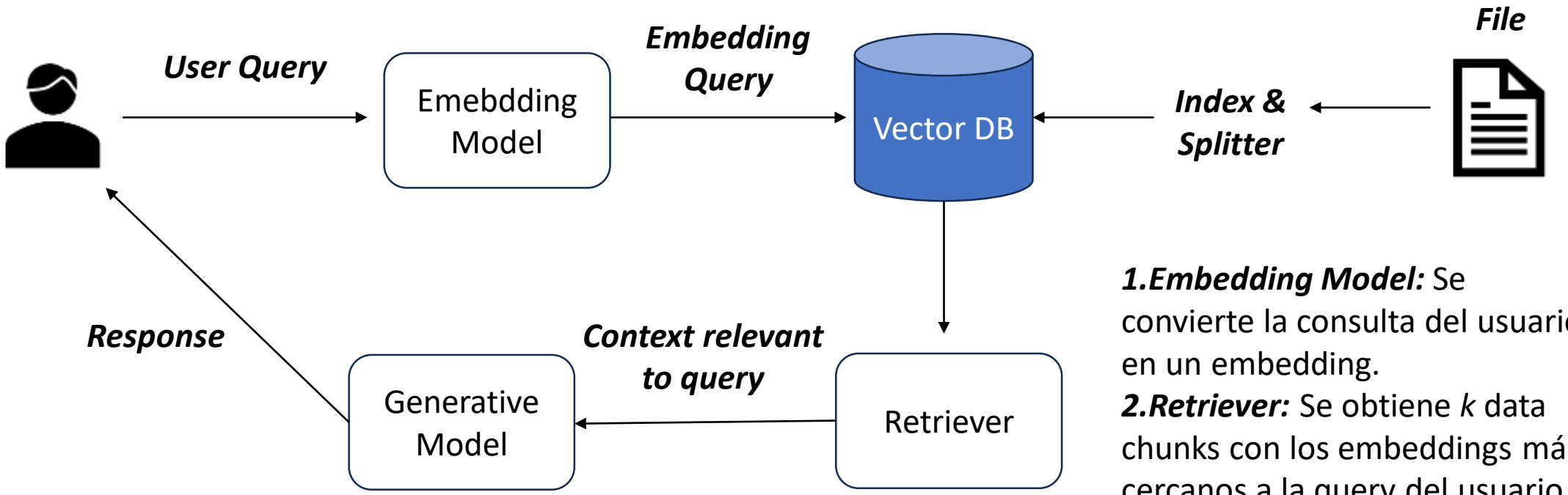
El LLMs seria alimentado tanto con la información de la arquitectura de los LLMs, junto con la película de Transformers.

¿Qué pasaría si solo usamos term base?



Hybrid Search:
Lo mejor de los dos mundos.

Arquitectura de RAG:



1.Embedding Model: Se convierte la consulta del usuario en un embedding.
2.Retriever: Se obtiene *k* data chunks con los embeddings más cercanos a la query del usuario.

Divide and Conquer: *Divide el texto en fragmentos.*

Large language models (LLMs) are a type of artificial intelligence algorithm that uses deep learning techniques and massively large data sets to understand, summarize, generate and predict new content. The "large" in LLM refers to the vast amount of data these models are trained on – often terabytes of text and data – and the number of parameters they use to process language, which can be billions.

LLMs are built on transformer architectures, a neural network architecture introduced in 2017 that processes input data in a way that allows for parallelization, making training on massive data sets feasible. This architecture uses self-attention mechanisms to weigh the importance of different words in the input sequence when processing a specific word.

There are different types of LLMs, categorized based on their architecture and training objectives:

* **Generative LLMs:** These models are designed to generate new content, such as text, code, or images. Examples include GPT-3, LaMDA, and Stable Diffusion.
* **Discriminative LLMs:** These models are used for classification and analysis tasks, such as sentiment analysis or named entity recognition.
* **Hybrid LLMs:** These models combine aspects of both generative and discriminative models.

Chunking is a crucial step in preparing text data for LLMs, especially for tasks like RAG. It involves breaking down a large text document into smaller, manageable pieces or "chunks." The size and method of chunking can significantly impact the performance of LLM applications.

*Los archivos son divididos en **chunks,** lo que permite localizar y recuperar únicamente la información relevante para cada consulta.*

1. Chunk size — el tamaño de cada fragmento (medido en tokens o caracteres). Define cuánto texto va en cada chunk. Si es muy grande pierdes precisión en el retrieval; si es muy pequeño pierdes contexto.

2. Chunk overlap — cuántos tokens/caracteres se repiten entre un chunk y el siguiente. Sirve para evitar que una idea importante quede cortada justo en el borde entre dos chunks.

Los LLMs se pierden con infinito contexto.

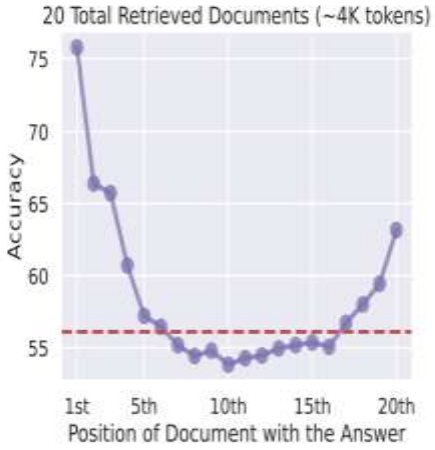
Lost in the Middle: How Language Models Use Long Contexts

Nelson F. Liu[1*] Kevin Lin[2] John Hewitt[1] Ashwin Paranjape[3]
Michele Bevilacqua[3] Fabio Petroni[3] Percy Liang[1]
[1]Stanford University [2]University of California, Berkeley [3]Samaya AI
nfliu@cs.stanford.edu

20 Nov 2023

Abstract

While recent language models have the ability to take long contexts as input, relatively little is known about how well they *use* longer context. We analyze the performance of language models on two tasks that require identifying relevant information in their input contexts: multi-document question answering and key-value retrieval. We find that performance can degrade significantly when changing the position of relevant information, indicating that current language models do not robustly make use of information in



length of 4K tokens, and GPT-3.5-Turbo (16K) is a version with an extended maximum context length of 16K tokens. We evaluate Claude-1.3 and Claude-1.3 (100K) with the Anthropic API; Claude-1.3 has a maximum context length of 8K tokens, and Claude-1.3 (100K) has an extended context length of 100K tokens. [6]

2.3 Results and Discussion

We experiment with input contexts containing 10, 20, and 30 total documents. Figure 5 presents multi-document question answering performance when varying the position of relevant information within the input context. To contextualize model performance, we also evaluate on the closed-book and oracle settings (Table 1). In the closed-book setting, models are not given any documents in their input context, and must rely on their parametric memory to generate the correct answer. On the other hand, in the oracle setting, language models are given the single document that contains the answer and must use it to answer the question.

Model performance is highest when relevant information occurs at the beginning or end of its input context. As illustrated in Figure 5, changing the position of relevant information in the input context leads to substantial decreases in model performance. In particular, we see a distinctive U-shaped performance curve—models are often much better at using relevant information that occurs at the very beginning (primacy bias) and very end of contexts (recency bias), and suffer degraded performance when forced to use information within the middle of its input context. For example, GPT-3.5-Turbo's multi-document QA performance can drop by more than 20%—in the worst case, performance in 20- and 30-document settings is lower than performance without *any* input documents (i.e., closed-book performance; 56.1%). These results indicate that current models cannot effectively reason over their entire context window when prompted for downstream tasks.

Extended-context models are not necessarily better at using input context. When the input context fits in the context window of both a model and its extended-context counterpart, we see that performance between them is nearly identical. For example, the 10- and 20-document settings both fit in the context window of GPT-3.5-Turbo and GPT-3.5-Turbo (16K), and we observe that their performance as a function of position of relative information is nearly superimposed (solid purple and dashed brown series in Figure 5). These results

[5]We use the `0613` OpenAI model versions.

[6]We also evaluate GPT-4 (8K) on a subset of multi-document QA experiments, finding similar results and trends as other models (though GPT-4 has higher absolute performance). Evaluating GPT-4 on the full multi-document QA and key-value retrieval experiments would cost upwards of $6000. See Appendix D for GPT-4 results and discussion.

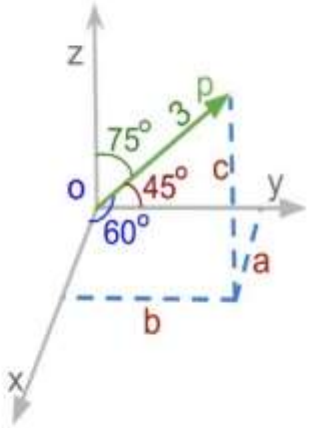
LongChat-13B (16K)	35.0%	83.4%
MPT-30B-Instruct	31.5%	81.9%
GPT-3.5-Turbo	56.1%	88.3%
GPT-3.5-Turbo (16K)	56.0%	88.6%
Claude-1.3	48.3%	76.1%
Claude-1.3 (100K)	48.2%	76.4%

Table 1: Closed-book and oracle accuracy of language models on the multi-document question answering task.

https://arxiv.org/pdf/2307.03172

Similitud de Coseno: "Cercanía" en los chunks.

$$similitud(\vec{a}, \vec{b}) = \frac{\vec{a} \cdot \vec{b}}{\|\vec{a}\| \|\vec{b}\|}$$



La similitud de coseno es la manera para determinar algebraicamente si dos vectores son similares o no.

El resultado va de **-1 a 1**:

- **1** = vectores idénticos (misma dirección)
- **0** = vectores perpendiculares (sin relación)
- **-1** = vectores opuestos

Los embeddings de texto producen vectores cuyos valores de similitud coseno van de 0 a 1 en la práctica, donde valores cercanos a 1 indican mayor similitud semántica entre los textos comparados.

¿Qué falta aquí?

¿Qué falta aquí?

Lo que se mide, se gestiona. *Peter Drucker (1909-2005)*

Control: *Métricas de Recuperación y Generación.*

Retrieval

Generation

Recall

De los documentos relevantes que existen en la base, ¿Cuántos recuperaste?

Etiquetar documentos

Precision

De los documentos recuperados, ¿cuantos son realmente relevantes?

AI Agent podría validar

Faithfulness

¿La pregunta se basa en el contexto proporcionado?

Una baja indica el modelo alucinada o inventa.

Answer Relevancy

¿La respuesta aborda de forma directa y útil la pregunta original?

Una respuesta puede ser fiel al contexto, pero no útil

RAGAS: Librería especializada para medir RAGs

Ragas Office Hours - If you need help setting up Evals for your AI application, sign up for our Office Hours here.

Ragas | Search | vibrantlabsai/ragas ☆ 13k ⑂ 1.3k

Get Started | Core Concepts | How-to Guides | References | Community

Introduction

Copy page

Ragas is a library that helps you move from "vibe checks" to systematic evaluation loops for your AI applications. It provides tools to supercharge the evaluation of Large Language Model (LLM) applications, enabling you to evaluate your LLM applications with ease and confidence.

Why Ragas?

Traditional evaluation metrics don't capture what matters for LLM applications. Manual evaluation doesn't scale. Ragas solves this by combining **LLM-driven metrics** with **systematic experimentation** to create a continuous improvement loop.

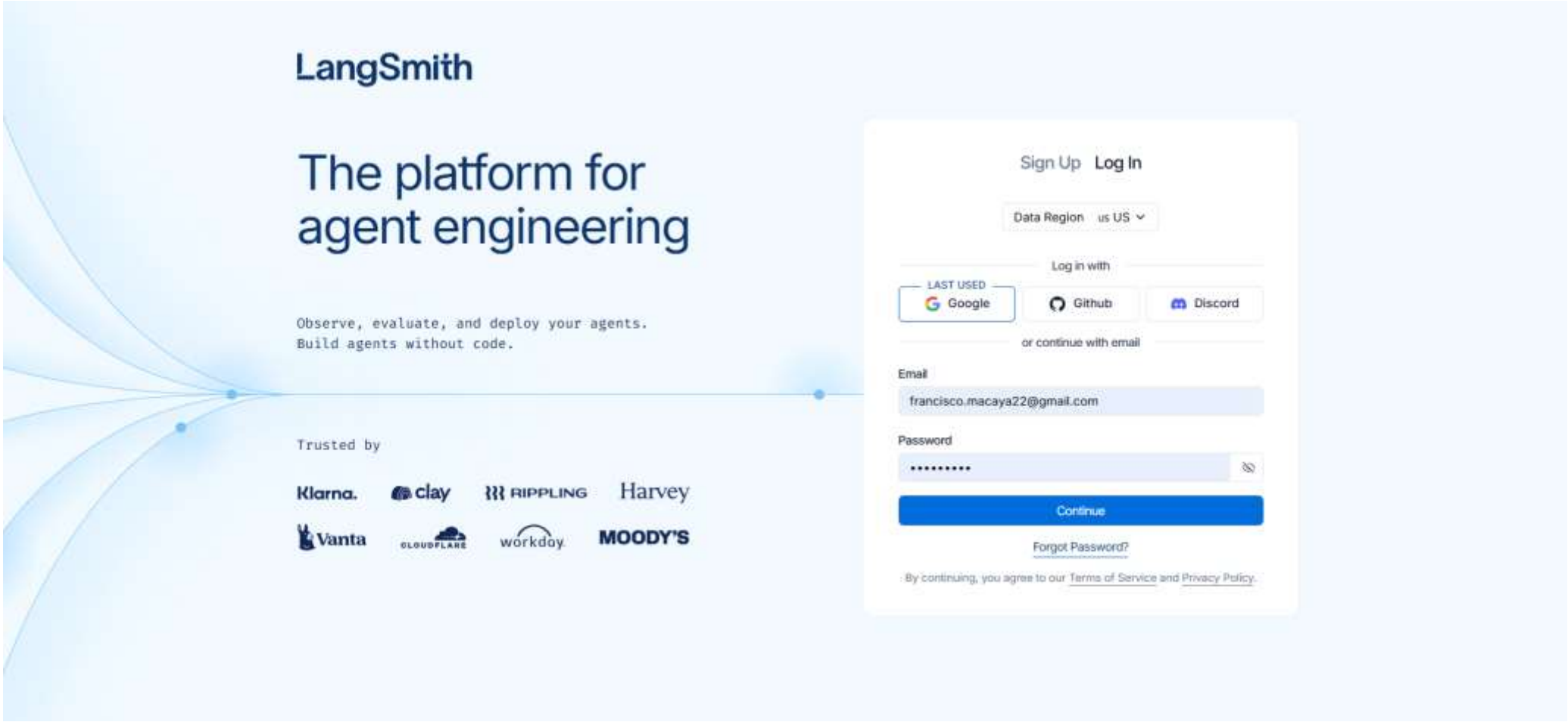
Key Features

- **Experiments-first approach**: Evaluate changes consistently with `experiments`. Make changes, run evaluations, observe results, and iterate to improve your LLM application.
- **Ragas Metrics**: Create custom metrics tailored to your specific use case with simple decorators or use our library of available metrics. Learn more about metrics in Ragas.
- **Easy to integrate**: Built-in dataset management, result tracking, and integration with popular frameworks like LangChain, LlamaIndex, and more.

Table of contents

Why Ragas?
Key Features
Want help improving your AI application using evals?

https://docs.ragas.io/en/stable/

Todo parece bien, pero … como mido realmente. Necesito registros de buggs, errores, llamadas a las Tools, vector databases , etc.

LangSmith: Todo en un lugar.